

Laboratory Activity 1 - Academic Report

Real-Time Multisensor Room Monitoring System

Course: BCA182 Embedded Systems Programming

Activity: Laboratory Activity No. 1 (100 points)

Repository: `bca182-freertos-multisensor` (local Git, milestones Mo-M16)

Target: STM32 Blue Pill STM32F103C8T6, simulated in Wokwi

Toolchain: PlatformIO + STM32Cube (HAL/CMSIS) + native FreeRTOS, C++

Date: 28 September 2026

This report is the academic deliverable required by sections 57-60 of the laboratory specification. It is deliberately separate from the public `README.md`: the README showcases the finished project, while this document records the reasoning, the measurements and the evidence behind each design decision. Section numbers in square brackets refer to `../Laboratory 1/BCA182 - Laboratory Activity 1.md`; file and line references point at the committed source, so every claim below can be checked against the repository.

1. Problem and Requirements

1.1 The problem

The laboratory asks for a *simulated STM32 room-monitoring system*: a device that watches room/environmental conditions and lets a user inspect them locally. The concrete problem is a room monitor with four sensing inputs (temperature, humidity, ambient light, motion) and three outputs (an OLED page display, a temperature alarm buzzer, a serial diagnostic log), where the hard part is not any single driver but the *concurrency structure*: six application tasks plus two diagnostics must coexist on one Cortex-M3 without starving each other, without unsynchronised shared state, and without turning into a loop of delays.

1.2 Functional requirements and how each is satisfied

Section 4 of the specification defines FR-01...FR-10, and the limits stated immediately after the table are **LOW = 18.0 degrees C**, **HIGH = 30.0 degrees C** (evaluated inclusively: `<= 18.0` is low, `>= 30.0` is high, `include/alarm.h`, `src/alarm.cpp`).

ID	Requirement	Required behaviour	Where it is implemented
FR-01	Temperature measurement	periodically obtain temperature	<code>src/sensors.cpp</code> , <code>SensorTask</code> , 2000 ms <code>vTaskDelayUntil()</code> period
FR-02	Humidity measurement	periodically obtain humidity	same DHT22 frame as FR-01, same task and period
FR-03	Ambient-light measurement	monitor relative ambient light	<code>src/sensors.cpp</code> , ADC1_IN0 single conversion + <code>ldr_to_light_percent()</code>
FR-04	Motion detection	detect simulated motion (PIR)	<code>src/motion.cpp</code> , <code>MotionTask</code> , PB8 polled every 10 ms
FR-05	OLED display	show one selected measurement at a time	<code>src/display.cpp</code> , <code>DisplayTask</code> (sole owner of I2C1), four pages
FR-06	Rotary encoder navigation	switch Temperature/Humidity/Light/Motion	<code>src/input.cpp</code> , <code>InputTask</code> , <code>nextDisplayMode()</code> / <code>previousDisplayMode()</code>
FR-07	Temperature alarm	buzzer outside the configured range	<code>src/alarm.cpp</code> , <code>AlarmTask</code> + pure <code>evaluateTemperature()</code>
FR-08	Activity state	ACTIVE and INACTIVE states	<code>src/system_state.cpp</code> , <code>StateTask</code> , two-state machine
FR-09	Automatic inactivity	no motion for a period -> INACTIVE	<code>StateTask</code> , <code>SYSTEM_INACTIVE_TIMEOUT_MS</code> = 15000 (the short laboratory timeout of section 31)
FR-10	Automatic reactivation	motion -> ACTIVE	<code>MotionTask</code> sets <code>EVENT_MOTION</code> , <code>StateTask</code> consumes it

The full traceability of every one of these rows - implementation file plus the specific verification evidence - is in **Appendix A** (section 60 matrix).

Beyond the ten FR rows, the specification makes FreeRTOS mandatory [section 6] and requires a specific concept list [section 9]: multiple tasks, explicit priorities, blocking delays, `vTaskDelayUntil()` on at least one periodic task, at least one queue, at least one mutex, at least one event group or task notification, a state machine, inter-task communication and shared-resource protection. Each of those has a *use* in this design, not merely a declaration - section 3 and Appendix A say which, and what would break if the mechanism were removed.

1.3 Required system behaviour in practice

- Six application tasks (`SensorTask`, `DisplayTask`, `InputTask`, `MotionTask`, `AlarmTask`, `StateTask`) plus the two section 17 diagnostic tasks (`TaskA`, `TaskB`) - eight tasks in total, created in `src/main.cpp:216-223`.
- Preemptive scheduling with round-robin time slicing, 7 priority levels, a 1 kHz tick (`include/FreeRTOSConfig.h`).
- Data flows over three length-1 overwrite queues, level state flows over a three-bit event group, and the one shared peripheral (USART1) is guarded by a mutex with priority inheritance (`include/rtos_objects.h`).
- Startup follows section 41's four stages inside `app_main()`: hardware initialisation -> FreeRTOS object creation -> task creation -> scheduler.

1.4 The Wokwi adaptation - why simulation changes the picture

The specification's system is a *simulated* system [section 3], and the difference between simulation and silicon is not cosmetic. Four properties of the real world disappear in Wokwi, and each one forced a documented adaptation:

1. **No real sensor noise.** The Wokwi DHT22 and LDR models return exactly the value last set on them: temperature sits at `25.40 C` forever unless a scenario changes it, and 500 lux always maps to `76 %` (`docs/functional-verification.md` section 3.1). The only "noise" ever observed is a 0.01 degrees C quantisation of the simulator's own model (two `25.39 C` samples in one run). Consequence: drift, jitter and filtering questions that a real sensor raises simply cannot be exercised; verification therefore focuses on *correct plumbing and correct decisions*, not on measurement quality. The firmware is still written as if the sensor were noisy - checksum validation, three attempts with a 100 ms retry gap (`SENSOR_READ_ATTEMPTS`, `SENSOR_RETRY_DELAY_MS` in `include/sensors.h`) - so the code does not depend on the simulator's tidiness.
2. **No audible buzzer.** The Wokwi buzzer part is a level indicator, not a tone generator: the alarm output is a static DC level on PBo. The functional record shows only two transitions in a 21.5 s run and a 6.000 s high time (`docs/functional-verification.md` section 3.3). The alarm is therefore verified as a *pin level with timing*, by logic-analyser VCD and by `expect-pin` assertions, never as sound; this report claims no acoustic property at all.
3. **No automation for the encoder and the PIR.** Wokwi's scenario engine supports `set-control` for the DHT22 and the photoresistor, but the `wokwi-ky-040` and `wokwi-pir-motion-sensor` parts silently ignore every control name tried (`rotate`, `rotation`, `step`, `clockwise`, `angle`, `motion`, `simulate-motion`, `trigger`). FT-04, FT-05, FT-08 and FT-10 therefore had to be driven *electrically* - momentary buttons on a temporary harness diagram producing the identical edge sequence on PB12/ PB13 and PB8 - and each of those rows carries an explicit disclosure that the firmware path is proven while the physical part interaction should still be repeated by hand in the Wokwi web UI.
4. **Constant simulated values, and an emulator with its own instruction quirks.** Because nothing moves unless a scenario moves it, "change something and watch the reaction" is the only available experimental method. And because Wokwi is an *emulator*, two of its instruction models are measurably wrong for this core (section 7.1): the firmware carries a documented, architecturally neutral port patch so that FreeRTOS can run at all. This is the one place where the submitted firmware knowingly deviates from a stock FreeRTOS port, and the deviation is argued in section 7.1.

Simulation also *reduces* risk in one direction: `pico run -t upload` is never needed, so the whole verification loop (build -> simulate -> observe serial and VCD) is reproducible on any machine with the Wokwi CLI.

2. System Architecture and Design

2.1 Hardware architecture

One MCU, six peripherals, one host serial link; `diagram.json` is the wiring of record and passes `wokwi-cli lint` with no issues.

Part	Wokwi part	Interface	Pin(s) / detail
MCU	<code>bluepill_f103c8</code>	-	72 MHz (HSE x9 PLL, HSI fallback)
DHT22	<code>wokwi:dht22</code>	1-wire, external 10 k pull-up	PA1 (open drain)
LDR	<code>wokwi:ldr</code>	ADC1_IN0, 12-bit	PA0
PIR	<code>wokwi:pir</code>	GPIO input, 10 ms poll	PB8
OLED	<code>wokwi:oled-ssd1306</code>	I2C1 master, address 0x3C	PB6 (SCL), PB7 (SDA)
Rotary encoder	<code>wokwi:rotary-encoder</code>	GPIO edge decode	PB12 (CLK), PB13 (DT), PB14 (SW, wired but unmapped)
Buzzer	<code>wokwi:buzzer</code>	GPIO output, active high	PB0
Host PC	serial monitor	USART1, 115200 8N1	PA9 (TX), PA10 (RX)

The onboard LED PC13 is driven high (off) once during initialisation and is never used again, so every active indicator in the system is either the OLED or the buzzer.

2.2 Software architecture

Section 40's required file list is implemented literally - one module per subsystem, with two extra headers (`FreeRTOSConfig.h`, `glcdfont.h`) that are technically justified in the source audit comment on `app_main()` (`src/main.cpp:68-80`):

```
include/  sensors.h display.h input.h alarm.h motion.h system_state.h
         rtos_objects.h FreeRTOSConfig.h glcdfont.h
src/      main.cpp sensors.cpp display.cpp input.cpp alarm.cpp motion.cpp
         system_state.cpp rtos_objects.cpp
```

Two architectural rules hold everywhere:

- **main.cpp stays at section 41's four stages.** It contains the clock tree, GPIO/serial bring-up, object creation, the eight `xTaskCreate` calls with the section 38/39 priority table next to them, and `vTaskStartScheduler()` - no sensor, display, alarm or state logic.
- **Pure decisions live at the centre, hardware at the edges.** `evaluateTemperature()` (`include/alarm.h`), `evaluateSystemState()` (`include/system_state.h`) and `nextDisplayMode()` / `previousDisplayMode()` (`include/input.h`) take values and return values: no HAL, no queue, no clock. This is what makes 29 desktop unit tests possible against the exact objects the firmware runs, with no `#ifdef UNIT_TEST` in production code (`test_build_src = yes` in `platformio.ini`).

`scripts/freertos_build.py` compiles the FreeRTOS kernel that PlatformIO's `stm32cube` builder omits, and links only `tasks.c`, `queue.c`, `list.c`, `event_groups.c` and `heap_4.c` - the rest of the middleware tree is excluded to keep flash usage down.

2.3 Subsystem decomposition

Each subsystem is a column: one Wokwi part -> one driver module -> one task -> the queue/bit it publishes to.

Subsystem	Driver module	Task	Publishes / consumes
Environment sensing	<code>sensors.cpp</code>	SensorTask (p2)	produces <code>displayQueue</code> , <code>alarmQueue</code>
Light display	<code>display.cpp</code>	DisplayTask (p1)	consumes <code>displayQueue</code> , <code>modeQueue</code> , reads all three event bits
User input	<code>input.cpp</code>	InputTask (p3)	produces <code>modeQueue</code>
Motion	<code>motion.cpp</code>	MotionTask (p3)	sets <code>EVENT_MOTION</code>
Alarm	<code>alarm.cpp</code>	AlarmTask (p2)	consumes <code>alarmQueue</code> , sets/clears <code>EVENT_ALARM</code> , drives PB0
System state	<code>system_state.cpp</code>	StateTask (p3)	consumes <code>EVENT_MOTION</code> (clear-on-exit), sets/clears <code>EVENT_ACTIVE</code>
Diagnostics	<code>main.cpp</code>	TaskA, TaskB (p1)	serial lines only (section 17)
Shared serial	<code>rtos_objects.cpp</code>	-	<code>serial_write()</code> choke point + <code>serialMutex</code>

Ownership is the synchronisation strategy for peripherals: `DisplayTask` is the only task that touches I2C1 [section 26], and `serial_write()` is the only path to USART1 [section 36]. There is no I2C lock, and no caller-side locking of the UART.

2.4 State machine

Two states, two transitions, one pure decision function [sections 32-34]:

- **ACTIVE -> INACTIVE:** `StateTask` waits on `EVENT_MOTION` with a 15 s timeout; on timeout with no motion, `evaluateSystemState()` returns `INACTIVE`, `EVENT_ACTIVE` is cleared, the OLED is blanked once and encoder detents are consumed and ignored. Sampling, PIR polling and the alarm keep running - only the presentation goes quiet.
- **INACTIVE -> ACTIVE:** `MotionTask` sets `EVENT_MOTION`, the wait returns, `evaluateSystemState()` prefers motion over timeout, `EVENT_ACTIVE` is set and the OLED is restored.

Boot starts `ACTIVE` and prints `State: ACTIVE` before any consumer can read a stale event group. Because prints happen only on a real transition, the serial `State:` lines are themselves test observables (FT-09, FT-10).

3. FreeRTOS Architecture

3.1 Kernel configuration

`include/FreeRTOSConfig.h`: `configUSE_PREEMPTION = 1`, `configUSE_TIME_SLICING = 1`, `configMAX_PRIORITIES = 7`, 1000 Hz tick, 9 KB heap (`heap_4`), mutexes enabled, stack-overflow checking method 2, malloc failed hook, `configASSERT` routed to `vAssertCalled()`. Time slicing and preemption together are what make the shared priority levels below safe to share: equal-priority ready tasks rotate on every tick.

3.2 Required FreeRTOS task table (section 59)

Eight tasks - the five in the specification's example plus `StateTask` (recommended by section 7, and present in our code) and the two section 17 diagnostic tasks. All priorities, periods and IPC objects are read from the source; they are the same values recorded in `src/main.cpp` and in the README's *Task Design* table.

Task	Responsibility	Trigger / Period	Priority	IPC	Typical Blocked Condition
SensorTask	Read DHT22 + LDR, publish <code>SensorData</code>	every 2000 ms (<code>vTaskDelayUntil</code>)	2	produces <code>displayQueue</code> , <code>alarmQueue</code> (length-1 <code>xQueueOverwrite</code>)	Blocked (delayed) on <code>vTaskDelayUntil</code> until the next period
DisplayTask	Render one OLED page; sole owner of I2C1	woken by a new sample or page; also re-checks every 50 ms	1	consumes <code>displayQueue</code> , <code>modeQueue</code> ; reads <code>EVENT_ACTIVE</code> / <code>EVENT_ALARM</code>	Blocked on <code>xQueueReceive(displayQueue, 50 ms)</code> - a bounded wait, not <code>portMAX_DELAY</code>
InputTask	Debounce and decode the rotary encoder, publish the page	every 2 ms (<code>vTaskDelay</code>)	3	produces <code>modeQueue</code> (length-1 <code>xQueueOverwrite</code>)	Blocked (delayed) 2 ms after every poll
MotionTask	Poll PIR, set <code>EVENT_MOTION</code> while OUT is high	every 10 ms (<code>vTaskDelay</code>)	3	sets <code>EVENT_MOTION</code> in <code>systemEvents</code>	Blocked (delayed) 10 ms after every poll
AlarmTask	<code>evaluateTemperature()</code> , drive buzzer PB0, publish <code>EVENT_ALARM</code>	on each published sample	2	consumes <code>alarmQueue</code> (blocking); sets/clears <code>EVENT_ALARM</code>	Blocked on <code>xQueueReceive(alarmQueue, portMAX_DELAY)</code> - idle between samples
StateTask	ACTIVE/INACTIVE machine (ours - not in the section 59 example table)	event-driven with a 15000 ms inactivity timeout	3	consumes <code>EVENT_MOTION</code> (clear-on-exit wait); produces <code>EVENT_ACTIVE</code>	Blocked on <code>xEventGroupWaitBits(EVENT_MOTION, 15 s timeout)</code>
TaskA	Diagnostic line (section 17)	every 1000 ms	1	none (serial line only)	Blocked (delayed) 1000 ms
TaskB	Diagnostic line (section 17), 500 ms start offset	every 1000 ms	1	none (serial line only)	Blocked (delayed) 1000 ms

3.3 Per-row justification

The specification asks [section 59] for technical justification rather than a reproduced checklist, and [section 39] for reasoning in terms of *prompt response*, *tolerated latency* and *what fails if the priority is wrong*. The per-task version of that argument lives as one table in `src/main.cpp:104-215` next to the `xTaskCreate` calls it applies to; what follows is the same argument read row by row, including what each IPC choice buys.

SensorTask (2 s period, priority 2, two length-1 queues). It is the system's only periodic producer, and its data is by definition up to one period old, so its urgency equals its own period: placing it at 2 means an alarm decision never waits behind a redraw, while a DHT22 frame plus formatting cannot preempt the level-3 one-shot tasks. The IPC choice - `xQueueOverwrite` on length-1 queues - buys "latest sample wins": a consumer that is busy for 100 ms never stalls the producer and never receives stale data afterwards. It blocks on `vTaskDelayUntil()` (section 4 below), i.e. on its own next period.

DisplayTask (event/50 ms, priority 1, bounded queue receive). Section 7 sets no deadline for a pixel, which is the definition of "can tolerate latency"; its cost per activation is the largest unprivileged stretch in the system (the ~100 ms I2C flush of the 1024-byte framebuffer), so putting it at the bottom is what keeps that flush from ever delaying sensing or alarming. The IPC it consumes is the same overwrite queues, plus `modeQueue` with a 0 ms receive, and the three event bits read non-destructively with `xEventGroupGetBits()`. It blocks on a **bounded** 50 ms receive rather than `portMAX_DELAY`: that is a milestone 8 design decision documented at `src/display.cpp:54-66` - once page turns also have to reach the screen, an unbounded wait would sleep through them, while a 50 ms bound still means the task is asleep between events (section 19 holds) and puts every turn on the OLED well inside the "within a second" window of FT-04/FT-05. Re-rendering happens only when something changed, so the extra wake-ups cost one queue check.

InputTask (2 ms poll, priority 3, queue). A knob detent is an edge that exists only while the pins move - it cannot be replayed the way a missed sample can, so this is "prompt response" work; below level 3 a page turn would land behind a 2 s sensor cycle or a 100 ms flush, above it a two-pin read would preempt the state machine for no benefit. Its IPC is a **queue** (`modeQueue`), which deliberately differs from the specification example's "notification/queue": a task notification carries no payload, so "latest page wins" would have needed a shared variable plus a lock, whereas a length-1 overwrite queue gives the same newest-wins semantics as the sensor channels and lets `DisplayTask` - the only task allowed to touch the OLED - decide when to re-render. It blocks on a 2 ms `vTaskDelay()` after every poll, which is also why fault experiment 1 (section 5) starves the system when that single line is deleted.

MotionTask (10 ms poll, priority 3, event group). The PIR's OUT pulse is the only copy of the evidence, so a missed edge cannot be replayed either; too low a priority would mean a person entering the room does not reach the state machine until the next 2 s sensor sample, trailing reality by a whole sample period. It only ever sets `EVENT_MOTION` - nobody waits on a holder, so there is no inversion question. It blocks on a 10 ms delay after each read.

AlarmTask (queue-driven, priority 2, blocking queue receive). Its input changes only every 2 s, so that is its entire latency budget: level with the producer buys nothing earlier (the data cannot arrive sooner) and costs preemption of the one-shot tasks; below

the producer an out-of-range temperature would latch *after* the OLED redraw instead of before. Its IPC is a dedicated `alarmQueue` so that it and `DisplayTask` cannot steal each other's samples (section 25's one-queue-per-consumer rule), and it blocks with `portMAX_DELAY` - a genuinely idle resource between samples, so the wait is free.

StateTask (event-driven with 15 s timeout, priority 3, event group wait). This task is ours: section 7 only *recommends* it, and the section 59 example table omits it, but a central ACTIVE/INACTIVE owner is what lets the display and the encoder gate on a single bit instead of each re-implementing the timeout. It turns motion evidence into the result other tasks gate on, so it runs at the same level as its producer: too low and a transition queues behind the 2 s sample (OLED ~2 s too long lit in an empty room), too high and it buys nothing - its wait is one timeout-bounded `xEventGroupWaitBits()`. It blocks on that wait: motion wakes it early, otherwise the 15000 ms timeout fires and it evaluates the transition.

TaskA / TaskB (1000 ms, priority 1, no IPC). Section 17 diagnostics: nobody acts on the line, so unbounded latency is acceptable - section 39's "can tolerate latency" class - which is the lowest level in the system. TaskA was lowered from priority 2 in milestone 12: at 2 it could preempt an OLED flush in progress to print a demo line, a cost with no benefit (exactly the poorly-selected priority section 39 asks students to name). The pair shares level 1 with `DisplayTask` and cannot starve it: one line per second each, microseconds of CPU, and TaskB's 500 ms start offset keeps them half a period apart forever. They block on a 1000 ms `vTaskDelay()`.

Shared levels cannot starve each other. At 3 every task returns to sleep after microseconds of work (2 ms poll, 10 ms poll, one event wait); at 2 `AlarmTask` sits in `xQueueReceive`; at 1 `DisplayTask` waits on its queues and the diagnostics block for 1000 ms. Round-robin time slicing only ever divides idle time between equal-priority ready tasks.

3.4 Where our values intentionally differ from the specification's example

Row	Specification example	Ours	Why
<code>InputTask</code> IPC	"Notification/queue"	length-1 queue (<code>modeQueue</code>)	payload semantics without a shared variable; see 3.3
<code>DisplayTask</code> blocked on	"Waiting for data"	bounded 50 ms receive	milestone 8: page turns must also wake it (<code>src/display.cpp:54-66</code>)
<code>SensorTask</code> blocked on	"Delay"	<code>vTaskDelayUntil()</code>	section 22/23: fixed phase, no drift
<code>MotionTask</code> IPC	event group	event group (<code>EVENT_MOTION</code>)	unchanged
<code>AlarmTask</code> IPC	queue	queue (<code>alarmQueue</code> , <code>portMAX_DELAY</code>)	unchanged
rows present	5 tasks	8 tasks	adds <code>StateTask</code> (section 7) and <code>TaskA / TaskB</code> (section 17)

Every priority in our table equals the specification's example values (2, 1, 3, 3, 2) for the five rows it lists.

3.5 Task-state analysis

Section 18 asks students to distinguish Running, Ready and Blocked in their own implementation. Three snapshots of the real system:

Snapshot A - steady state between samples (the common case). The scheduler runs the idle task (priority 0, Running). All eight application tasks are **Blocked**: `SensorTask` on `vTaskDelayUntil` (2 s period), `AlarmTask` on `xQueueReceive(alarmQueue, portMAX_DELAY)`, `DisplayTask` on its 50 ms bounded receive, `StateTask` on `xEventGroupWaitBits` with the 15 s timeout, `TaskA / TaskB` on their 1000 ms delays, `InputTask` on its 2 ms delay, `MotionTask` on its 10 ms delay. The only application tasks that ever leave that Blocked set between samples are `InputTask` and `MotionTask`, and only for the microseconds of their run window at a poll boundary - so the steady state of this system is "everything blocked, idle running", which is the healthy condition section 19 asks for: every task blocks after finite work.

Snapshot B - the sample instant (t = 2000 ms boundary). `SensorTask`'s delay expires, so it moves Blocked -> Ready, and as the highest-priority Ready task (2) it becomes Running. For roughly 5 ms it holds the CPU with `vTaskSuspendAll()` active while the DHT22 answer frame is timed on the DWT cycle counter - during that window a level-3 task may become Ready (its delay expired) but cannot run: it stays Ready, and the accumulated wait is the worst-case latency level 3 ever eats (~5 ms plus formatting/print time). When `SensorTask` publishes, `xQueueOverwrite(displayQueue, ...)` and then `xQueueOverwrite(alarmQueue, ...)` (`src/sensors.cpp:445-446`) make `DisplayTask` and `AlarmTask` Ready. `AlarmTask` (2) preempts `DisplayTask` (1) as soon as `SensorTask` blocks, evaluates the alarm and prints within about 2 ms of the sample - this ordering is precisely what fault experiment 2 disrupts (section 5.3). `DisplayTask` then runs when the higher levels sleep and spends ~100 ms flushing the framebuffer. `InputTask / MotionTask / StateTask` keep cycling Ready (microseconds) -> Blocked (2 ms / 10 ms / 15 s) as before.

Snapshot C - an encoder detent (FT-04). `InputTask` reads the edge on its next 2 ms poll (Running), passes the two-sample debounce, overwrites `modeQueue` (which never blocks the producer), prints `Page: ...` under `serialMutex`, then blocks again. `DisplayTask` becomes Ready, runs once the level-3 and level-2 tasks are asleep, drains `modeQueue` with a 0 ms receive and re-renders. The whole response is bounded by one 2 ms poll plus the time higher-priority tasks need - not by the 50 ms poll.

Suspended and Deleted [learning outcome 7]: no task in this system is ever suspended or deleted. All eight live from boot to power-off, which is a deliberate choice - dynamic task lifetime would spend the heap for no benefit on a fixed function set. The one *suspension* in

the code is `vTaskSuspendAll()` around the DHT22 frame in `src/sensors.cpp:280`; that is the **scheduler** being suspended for ~5 ms so a preemption cannot stretch a time-critical waveform, not a task entering the Suspended state, and it ends before any serial print.

3.6 Communication and synchronisation

Object	Type	Producers	Consumers	Semantics
displayQueue	queue, length 1	SensorTask	DisplayTask	latest SensorData wins; receive waits <= 50 ms
alarmQueue	queue, length 1	SensorTask	AlarmTask	same sample, second consumer, portMAX_DELAY
modeQueue	queue, length 1	InputTask	DisplayTask	latest DisplayMode, 0 ms receive
systemEvents EVENT_ACTIVE (BIT0)	event bit	StateTask	DisplayTask, InputTask	mirrors the machine state; readers use xEventGroupGetBits() and never clear it
systemEvents EVENT_MOTION (BIT1)	event bit	MotionTask	StateTask	level while PIR is high; only StateTask clears it (wait with clear-on-exit)
systemEvents EVENT_ALARM (BIT2)	event bit	AlarmTask	DisplayTask	non-NORMAL temperature; appended to the OLED value line
serialMutex	mutex	any task	any task	one atomic line on USART1, priority inheritance bounds inversion

Section 25's diagram shows one sensor queue feeding two consumers. A queue hands each item to exactly one receiver, so a *single* shared queue would let DisplayTask and AlarmTask race for every sample and each see only half the stream; the design therefore uses **one queue per consumer carrying the same message** (documented on `include/rtos_objects.h:35-49`).

The event group carries *levels*, not messages: ACTIVE/INACTIVE and ALARM are levels, so `xEventGroupGetBits()` lets a reader check them without blocking and without consuming the producer's evidence. Only `EVENT_MOTION` is edge-like, so `StateTask` is its single consumer and takes it with clear-on-exit.

The mutex protects USART1 - the shared resource, the competing tasks and the failure mode are documented in full on `include/rtos_objects.h:112-191` (section 37 material). In brief: the resource is the UART handle and its `gState` state machine; the competitors are every `serial_write()` caller (eight tasks plus the boot banner); the failure mode is `HAL_UART_Transmit()` returning `HAL_BUSY` to a second caller, which - because every caller ignored that return - silently dropped the loser's whole line. The mutex is taken once around the whole transmit, so one call = one atomic line. It is a mutex and not a binary semaphore because only a mutex has an owner, and ownership is what switches on **priority inheritance**: if `InputTask` (3) waits while `SensorTask` (2) holds the UART, the holder is boosted for the duration of the critical section, so an unrelated mid-priority task cannot stretch the wait - the unbounded inversion a binary semaphore would allow. `portMAX_DELAY` is safe there because no caller prints from an ISR (fault paths use `serial_write_fault()`, which bypasses both HAL and mutex) and no caller holds a critical section or a scheduler suspension across a print.

4. Implementation

Significant decisions only, each attributed to the document or source comment that records it; this is not a line-by-line narration.

Wokwi port workaround: yield through SVC, cpsie replaced by msr primask. The stock Cortex-M3 port yields from thread mode with a bare store to ICSR (PENDSVSET). On silicon the hardware stacks the address of the *next* instruction; the Wokwi STM32F103 model services PendSV while the store is still executing and stacks the address of the *store itself*, so a task that has just called `vTaskDelay()` and is still the highest-priority ready task resumes on the yield instruction and re-pends forever. The fix (`include/FreertosConfig.h:149-189` plus the `WOKWI_PORT_PATCH` in `scripts/freertos_build.py:69-135`) redefines `portYIELD_WITHIN_API()` as `svc 0` and makes `vPortSVCHandler()` perform the context switch itself - save `r4-r11` plus `pxTopOfStack`, call `vTaskSwitchContext()`, restore the winner, return straight to thread mode - so there is no second exception and therefore no stacked PC for the emulator to get wrong; a yield taken inside a critical section is deferred to `vPortExitCritical()`. A second patch replaces `cpsie i/cpsie f` in `prvPortStartFirstTask()` with `msr primask, r0 / msr faultmask, r0`, because the emulator decodes the CPS enable/disable halves inverted (section 7.1). Both substitutions are architecturally identical on real hardware, the patch is re-derived from the vendor file on every build, and the script aborts if the expected vendor text is missing, so a framework update cannot silently drop it.

-w1,-u,-printf_float for newlib-nano (`platformio.ini:59`). newlib-nano omits `%f` unless the float formatter is forced in; both the OLED and the serial line print floats. The alternative - hand-rolled fixed-point - would have spread rounding decisions through the display code; this costs some flash and keeps one formatting path honest (README *Engineering Decisions*).

DHT22 start pulse and bus release. The host start signal is held low for **10 ms** (`DHT22_START_LOW_US 10000`, `src/sensors.cpp:49-51`), well above the datasheet's ≥ 1 ms, and the line is released by writing `ODR = 1` *first* and only then switching to input mode (`src/sensors.cpp:282-287`) - otherwise the "release" would be a pull-down. Development measurement: with the earlier 1200 us pulse

the read failed about 40% of the time in simulation; at 10 ms every read succeeds (the comment records exactly this). Read attempts retry up to 3 times with a 100 ms gap (`include/sensors.h:61-62`) as belt and braces. Section 7.2 covers the investigation, including the rejected hypotheses.

Length-1 overwrite queues, one per consumer (section 25; documented on `include/rtos_objects.h:35-63`). Only the newest sample is worth rendering or alarming on, and overwrite semantics remove the slow-consumer hazard entirely: `DisplayTask`'s ~100 ms flush cannot stall `SensorTask` because the queue is never full.

Single-owner OLED instead of a display lock [section 26]. One task, one bus, zero mutexes around I2C1: ownership is the synchronisation. The framebuffer is private to `display.cpp`.

Serial mutex at the single choke point [section 36, section 37]. All serial output funnels through `serial_write()` in `rtos_objects.cpp`, so the take/give pair exists once and no caller can forget it; a pre-scheduler guard transmits directly before `rtos_objects_create()` so the boot banner still appears, and kernel fault paths bypass the mutex on purpose (a non-recursive mutex taken from an exhausted stack or from `configASSERT` could deadlock exactly when the system is already broken).

`vTaskDelayUntil()` in `SensorTask` [section 22, section 23]. `src/sensors.cpp:448` wakes on absolute instants (`last_wake_time + 2000 ms`), not on "now + 2000 ms". `vTaskDelay()` measures from the moment the task becomes ready again, so the execution time of the body (a DHT22 frame, an ADC conversion, formatting, two queue overwrites - variable and retry-sensitive) is added to every period and the sample instants drift unboundedly; `vTaskDelayUntil()` skips the elapsed part and re-anchors to the schedule, so the period is exactly 2000 ms regardless of how long the read took, and consumer-side behaviour (alarm latency, redraw cadence, the 30 samples per 60 s seen in every reference run) is phase-stable.

Cycle-accurate timing with the DWT counter. The 1-wire answer frame needs microsecond edges; `HAL_GetTick()` is 1 ms and `HAL_Delay()` blocks, so the driver counts CPU cycles (72 cycles/us at 72 MHz) and suspends the scheduler only inside the ~5 ms frame, before any print.

Input poll period 2 ms (`src/input.cpp:30-49`). Wokwi documents the KY-040 pulses as "a few milliseconds" wide, so 100 Hz (10 ms) can miss a detent entirely; 1 kHz was measured first and *stalls the emulator* (less than one second of output in a 45 s run), while 2 ms and 10 ms complete a 45 s run. 2 ms is the fastest period that keeps the simulation real-time and still puts at least two samples inside every pulse for the two-consecutive-sample debounce.

Pure decision functions with unit tests against production sources. `test_build_src = yes` with `build_src_filter = + <alarm.cpp> + <input.cpp>`, and the state rules are header-only, so firmware and tests compile the same objects - the tests cannot drift from the code they test.

5. Verification and Testing

Three independent layers: host unit tests for the pure logic, functional tests in Wokwi for the end-to-end paths, and deliberate fault experiments for the FreeRTOS mechanisms themselves. The traceability matrix that ties every requirement to one of these is in **Appendix A** (placed as an appendix rather than inside this section, so section 5 stays about *how* we test and the matrix stays a flat, checkable table).

5.1 Unit tests

`pio test ->` **29 test cases, 29 passed, 0 failed** (3 suites, `native` environment), against the minimum of 13 required by section 43:

Category (section 43)	Minimum required	Required coverage	Our suite	Tests
Temperature alarm logic	5	below / exactly / normal / exactly / above the limits	<code>test_alarm</code>	10
Display navigation	4	forward/reverse transitions and wraparound	<code>test_navigation</code>	11
System state	4	ACTIVE no-timeout; ACTIVE timeout; INACTIVE no-motion; INACTIVE motion	<code>test_state</code>	8
Total	13			29

What they actually verify (not screenshots - the sources are committed under `test/`):

- `test_alarm` (10): `evaluateTemperature()` boundaries - 18.0 and 30.0 inclusive, values just inside and just outside, far-out-of-range values, each `AlarmState`, plus a guard that the limits really are the section 4 values (18.0/30.0) and NaN-free inputs.
- `test_navigation` (11): every forward and backward transition of the four-page ring, wrap at both ends, a full clockwise walk, a full counter-clockwise walk, and `previous` undoes `next` from every page.
- `test_state` (8): `evaluateSystemState()` - motion wins over timeout, timeout only transitions from ACTIVE, no event holds the state, the 15 s constant is the section 31 value, and a diagram walk of inactivity then motion.

The suites run because a bare `pio test` walks `default_envs` (`bluepill_f103c8`, `native`) and the firmware environment carries `test_ignore = *`; the host stubs under `test/stubs/` exist only so the STM32 HAL and FreeRTOS includes resolve on a desktop - nothing links against them, and production code contains no `#ifdef UNIT_TEST`.

5.2 Functional verification in Wokwi (FT-01...FT-10)

Headless `wokwi-cli` runs with `--scenario-file`, `--serial-log-file`, `--vcd-file` and `--screenshot-file`; the record with line numbers and VCD timestamps is `docs/functional-verification.md`. Stimulus files and the harness diagram live outside the repository, so `diagram.json` was never modified.

ID	Stimulus	Observed (abridged)	Result
FT-01	DHT22 temperature 25.4 -> 28.0 -> 31.0 -> 25.4	Temperature: 25.40 -> 28.00 -> 31.00 -> 25.40 C; OLED shows 31.00 C ALARM	PASS
FT-02	DHT22 humidity 61.2 -> 45.0 -> 61.2	Humidity: 61.20 -> 45.00 -> 61.20 %; OLED 45.00 %	PASS
FT-03	LDR lux 500 -> 50 -> 500	Light level: 76 -> 38 -> 76 %; OLED 38 %	PASS
FT-04	4 clockwise detents on PB12/PB13	Page: Humidity -> Light -> Motion -> Temperature (wrap)	PASS, synthetic detents
FT-05	4 counter-clockwise detents	Page: Motion -> Light -> Humidity -> Temperature (wrap)	PASS, synthetic detents
FT-06	temperature 31.0 (> 30.0)	Alarm: HIGH_TEMPERATURE; buzzer PB0 rises at 6.252 s; expect-pin buzzer:2 == 1 passed	PASS
FT-07	temperature back to 25.4	Alarm: NORMAL; PB0 falls at 12.252 s (high exactly 6.000 s)	PASS
FT-08	PB8 high while ACTIVE	Motion: detected, no state change; the four page turns afterwards still work	PASS, synthetic PIR edge
FT-09	15 s with no motion	State: ACTIVE (line 3) -> State: INACTIVE (line 61)	PASS
FT-10	PB8 high while INACTIVE	Motion: detected immediately followed by State: ACTIVE; sampling resumes	PASS, synthetic PIR edge

Honest disclosure of synthetic stimuli. Four of the ten rows (FT-04, FT-05, FT-08, FT-10) were driven by scenario pulses on the *nets* rather than by the physical parts, because Wokwi exposes no automation control for the encoder and the PIR (section 1.4). The button signals are the same electrical event the real parts produce, so the firmware path (GPIO -> debounce -> edge decode -> event group -> queue -> output) is proven end to end and confirmed on the VCD (ENC_CLK first edge at 4.000 s, PIR_STIM pulses of 1.200 s and 6.000 s); what is *not* proven is the Wokwi element model of the knob and the PIR, and each row says so. Three additional OLED screenshots cover the display path directly. A further honesty note from the record: `wait-serial` was deliberately avoided as an observation mechanism because the runner's pause strips a CRLF and corrupts the log; all observation happens after the run, on the complete log file.

5.3 Deliberate fault experiments (sections 49-51)

Method (`docs/fault-experiments.md`): change **one** thing, rebuild, run 60 s headless, count every message shape against the reference, revert with `git checkout --`, prove `git diff --src include` is empty, rebuild. Only `src/input.cpp`, `src/main.cpp` and `src/rtos_objects.cpp` were ever touched. Reference run: **216 lines** with Task A/B 60/60, Temperature/Humidity/ Light 30/30/30, Page: 1, Alarm: 1, State: 1+1, 0 errors, longest line 27, 77 s of host wall clock per 60 s of simulated time.

Experiment	Change	Measured result	Reading
\$49 remove blocking	deleted the 2 ms <code>vTaskDelay()</code> at the end of <code>InputTask</code> 's loop	5 lines total (the two banner lines, <code>State: ACTIVE</code> , <code>Page: Temperature</code> , <code>State: INACTIVE</code>); Task A/B 0/0, all sensor output 0; host wall clock 115 s for the same 60 s	A ready priority-3 task that never sleeps permanently preempts every priority-1 and -2 task; nothing errors (ERR/assert/stack overflow all 0) - starvation is <i>silent</i> . The surviving application prints come from a pre-loop call (<code>Page:</code>) and from equal-priority time slicing of <code>StateTask</code> , which shares level 3.
\$50 change priority	<code>DisplayTask</code> from 1 to <code>configMAX_PRIORITIES - 1</code> (6)	line counts unchanged (84 in the controlled alarm scenario), but the buzzer rise moved 6.252 s -> 6.363 s (+111 ms) and the fall 12.252 s -> 12.364 s (+112 ms)	<code>SensorTask</code> publishes to the display <i>first</i> ; at priority 6 the queue write preempts immediately and <code>AlarmTask</code> (now below it) waits out the ~100 ms I2C flush. Responsiveness must be measured in <i>time</i> , not lines - a line count is blind to a 111 ms delay, and the task that lost margin is the one that can least afford it.
\$51 remove mutex	deleted the take/give pair from <code>serial_write()</code>	215 lines , reproducibly across two runs; <code>Page: Temperature</code> missing both times; 0 malformed/merged/truncated lines	The failure mode on this HAL is not interleaving but <i>silent whole-line loss</i> : <code>HAL_UART_Transmit()</code> returns <code>HAL_BUSY</code> to the loser without writing a byte, and the return value was ignored. Phase-stable task phases make the same line lose every run. Garbling is still possible in the narrow check-then-set window of <code>gState</code> , which is the second half of what the mutex rules out.

The three experiments are complementary: §49 turns delay into *starvation*, §50 turns priority into *latency*, §51 turns a shared resource into a *lost message*. After restoration, `pio run` (48.2%/55.4% - identical to baseline), `pio test` (29/29), `pio check` (0/0/35), `wokwi-cli lint` (clean) and a 60 s run (216 lines, every count identical to the reference) all reproduced, which is the section 66 "test results must be reproducible" requirement met by construction.

5.4 Reproducing the evidence

```
$ pio run          # 48.2% flash (31588/65536), 55.4% RAM (11348/20480)
$ pio test         # 29/29 (test_alarm 10, test_navigation 11, test_state 8)
$ pio check        # 0 high / 0 medium / 35 low
$ wokwi-cli lint diagram.json # No issues found
$ wokwi-cli . --timeout 60000 --timeout-exit-code 0 --serial-log-file run.log
                  # 216 lines, counts as in the reference table
```

A fresh 60 s run performed for this milestone (28 September 2026) reproduced the reference exactly: 216 lines, Task A/B 60/60 with no consecutive identical pairs, sensors 30/30/30, Page: 1, Alarm: 1, State: ACTIVE / INACTIVE 1/1 (INACTIVE at log line 60, i.e. after the 15 s timeout), banner exactly once, ERR /stack-overflow/assert 0/0, longest line 27, no empty lines.

6. Static Code Analysis

`pio check` with `cppcheck 2.11`, flags pinned in `platformio.ini` (`--inline-suppr`, `--enable=warning,style,performance,portability,unusedFunction`) so the run that produced the table stays reproducible.

Result: 0 high, 0 medium, 35 low (21 in the `bluepill_f103c8` pass, 14 in the `native` pass; both passes analyse the same `<src>` + `<include>` file set and differ only in headers and `-D` defines). Before triage there were 38 low-severity findings; nothing was suppressed with inline comments.

6.1 Findings that were fixed (corrective actions)

Finding	File/line	Cause	Corrective action
shadowFunction	src/display.cpp:223	<code>oled_char()</code> 's parameter <code>index</code> shadowed the POSIX function <code>index()</code> from <code><strings.h></code> , which the real Cube headers pull in (bluepill pass only)	renamed to <code>glyph_index</code> ; expression otherwise unchanged, so generated code is identical
constParameterPointer	src/input.cpp:171 (bluepill)	<code>publish_page(DisplayMode *mode)</code> only ever reads the page	parameter became <code>const DisplayMode *</code> , contract documented in the comment above it
constParameterPointer	src/input.cpp:171 (native)	same defect reported a second time because <code>input.cpp</code> is in both passes' file sets	fixed by the same edit

Both edits are behaviour-neutral - no value on any execution path changes - which is why the flash/RAM figures are unchanged after the fix. Those three rows are exactly the 38 -> 35 drop, and no finding was silenced with an inline `// cppcheck-suppress` comment: the tool was configured with `--inline-suppr` available, and it was never needed.

6.2 Interpreting the residual 35

Section 45 asks for interpretation, not command output. "Accepted" never means "ignored": every row in `docs/static-analysis.md` records why the line must stay as it is.

Class	Count	Why it is acceptable
Vendor/macro-mandated C-style casts	19	the cast lives inside a CMSIS or FreeRTOS macro (<code>I2C1</code> , <code>DWT</code> , <code>CoreDebug</code> , <code>configASSERT</code>) that must also compile as C for the kernel; a <code>static_cast</code> cannot be substituted into a macro the C kernel expands. Examples: <code>src/display.cpp:86</code> , <code>src/rtos_objects.cpp:81-85</code> , <code>src/sensors.cpp:232-313</code>
Signature-mandated parameters	6	the parameter type is fixed by a vendor or kernel prototype the file must match to link: <code>HAL_I2C_MspInit</code> , <code>HAL_ADC_MspInit</code> (called by name by the HAL), <code>vApplicationStackOverflowHook</code> (called by name from the kernel). Adding <code>const</code> would break the match
Constant-folding against the test stubs	8	host pass only: <code>xEventGroupGetBits()</code> and <code>HAL_GPIO_ReadPin()</code> in <code>test/stubs/</code> return compile-time constants, so conditions on them look provably fixed. On the target both are real data-dependent calls; the firmware pass reports none of these lines (<code>src/display.cpp:395-451</code> , <code>src/input.cpp:117,253</code> , <code>src/motion.cpp:151</code>)
Definitions whose prototype is visible	2	<code>unusedFunction</code> on <code>HAL_I2C_MspInit</code> / <code>HAL_ADC_MspInit</code> in the host pass: ST's headers declare them in the firmware pass, the stubs do not. They are not dead code - they enable the peripheral GPIO clocks

Two structural observations make the table readable instead of alarming: every `cstyleCast` and every genuine const-correctness issue is a *vendor* issue, and the split between passes is explained entirely by which headers each environment sees. The **bluepill pass is authoritative** for anything about vendor code. The residual 35 are therefore not "35 defects we chose to ignore" - they are 0 defects in application logic plus 35 places where C++ meets vendor C APIs or test stubs, each with a written reason.

7. Engineering Discussion

7.1 Debugging the Wokwi port: the PendSV freeze, the bounce, and `cpsie`

The freeze. The first integrated build with FreeRTOS hung before any task printed anything observable. The diagnosis was made on the emulator's register state rather than by printf-debugging: the PC was frozen at `vTaskDelay+0x0C` (the `str` instruction that writes `ICSR.PENDSVSET`), with `r3 = 0xE000ED04` and `r2 = 0x10000000` - i.e. the task was parked *on the yield itself*, re-pending PendSV forever. On silicon the same code is correct, because PendSV is taken after the store completes and the hardware stacks the *following* instruction; the Wokwi STM32F103 model enters PendSV while the store is still executing and stacks the store's own address, so whenever the yielding task is also the one PendSV would pick again (exactly the case after `vTaskDelay()` when no other task is ready) it resumes on the instruction that pends the exception.

The rejected first fix. The obvious repair was to move the ICSR store *inside* the SVC handler, keeping a two-exception design. That produced the second measured symptom: PendSV then returned into the SVC handler, so the two exceptions **bounced** against each other and no task context ever completed a switch (`scripts/freertos_build.py:94-112`). The only ICSR write proven safe in this emulator is the one made by `xPortSysTickHandler`, with BASEPRI held, taken on SysTick's own exception return.

The fix that stuck. Eliminate the ICSR write from thread mode entirely: `portYIELD_WITHIN_API()` (the documented `#ifndef` override point that every blocking API reschedules through) becomes `svc 0`, and the SVC handler performs the whole switch itself, branching on EXC_RETURN bit 2 - MSP path for the vendor first-task restore, PSP path for a task yield. `portYIELD()` is deliberately left alone: it is only reachable from ISR context, where the original store already behaves correctly, and an SVC raised from handler mode would fault. Yields inside a critical section are remembered in `wokwiYieldDeferred` and reissued at `vPortExitCritical()`, preserving the kernel's "never switch inside a critical section" rule.

The second emulator bug. `prvPortStartFirstTask()` ends with `cpsie i / cpsie f` before `svc 0`. Measured in simulation, the emulator decodes the CPS enable/disable halves inverted: `cpsid i` cleared PRIMASK and `cpsie i` set it, while `msr primask, r0` behaves correctly. Executed unpatched, `cpsie i` therefore *raises* PRIMASK immediately before `svc 0`, the SVC is pended forever, `prvPortStartFirstTask()` falls through and `xPortStartScheduler()` hits `prvTaskExitError()` - no task ever runs. The patch rewrites exactly that instruction pair as `msr primask, r0 / msr faultmask, r0`, which the emulator models correctly.

Why this deviation is defensible. It is *Wokwi-only in trigger, not in semantics*: CPSIE I/F and MSR PRIMASK,#0 / FAULTMASK,#0 are architecturally identical on a Cortex-M3, and an SVC that performs the switch is architecturally equivalent to a PendSV that does. The patch is applied to a copy of the vendor port at build time, is re-derived every build, and aborts if the expected vendor text is missing - so it cannot rot silently, and no vendor file in the repository was edited. Nothing in the application code knows the workaround exists. If this firmware is ever flashed to a physical Blue Pill, the same patch still compiles and still behaves correctly; what cannot be claimed is that it was *tested* there (section 7.6).

7.2 The DHT22 read-failure investigation

Early sensor integration produced frequent `DHT22 read failed` lines - roughly 40% of reads with the original code, which used the datasheet-minimum start pulse (`DHT22_START_LOW_US 1200`) and released the bus by switching the pin straight to input mode. Investigation, in the order it actually happened:

- **Hypothesis 1: inter-read interval.** The natural suspect was reading too often. Stretching the sample period to 3000 ms made the failure rate *worse*, not better - which kills the hypothesis: if the problem were recovery time between frames, a longer interval would have helped. This is the single most useful negative result of the investigation and the reason the report states it: a rejected hypothesis with a measured sign is worth more than three accepted ones without.
- **Hypothesis 2: start-pulse length.** The datasheet requires ≥ 1 ms; the Wokwi DHT22 model evidently wants more margin before it answers. Raising the pulse to **10 ms** (`src/sensors.cpp:49-51`) removed most of the failures. The comment in the source records the decision, including that the longer pulse is safe because holding the line low is a *register write* - it does not depend on uninterrupted execution - so the pulse runs with the scheduler live and only the time-critical answer is taken under the `vTaskSuspendAll()` window.
- **Hypothesis 3: the release ordering.** Releasing by writing `ODR = 1` and *then* switching to input mode matters: switching to input first would let the (absent) pull-down dominate and the sensor would never see the release. The current sequence (`src/sensors.cpp:282-287`) waits out the datasheet's 20-40 us after release before sampling the response.
- **Belt and braces.** `SENSOR_READ_ATTEMPTS = 3` with a 100 ms `SENSOR_RETRY_DELAY_MS` between them (`include/sensors.h:61-62`), so a residual transient costs at most one retry and never a missing line. In the current reference runs, `DHT22 read failed` never appears: 30/30/30 samples per 60 s.

The general lesson: the fix required *both* a margin change (10 ms) and a protocol-correctness change (release ordering); either alone left failures.

7.3 The serial `HAL_BUSY` line-drop defect and how the mutex proved it

Before the mutex existed, `HAL_UART_Transmit()` guarded itself with the handle state: a second caller arriving while `gState == BUSY_TX` was refused with `HAL_BUSY` **without writing a single byte**, and `serial_write()` ignored that return. The result was measured over 60 s: Humidity and Light level printed 29 times instead of 30, Task A running printed 56 instead of 60 (the 500 ms phase offset of the section 17 pair was *not* enough protection), and a State: ACTIVE line could disappear after a motion re-activation - seven missing lines, i.e. **209 of the reference 216** (the counts are recorded on `include/rtos_objects.h:148-164`, section 37 block). After the mutex was added at the single choke point, the same run produces the full 216.

Fault experiment 3 then re-derived the same defect from the other direction: removing the mutex from an otherwise unchanged build costs exactly one line (**215 vs 216**, twice), always Page: Temperature, with zero malformed lines. Two things follow, both of which section 51 asks for:

- the observed failure mode is **silent whole-line loss, not interleaving** - arguably worse, because a dropped line looks exactly like a task that never printed, while garbled text at least announces itself;
- interleaving is still possible in the handful of instructions between `gState == READY` and `gState = BUSY_TX`; 120 s of simulated time without the mutex happened not to hit that window, and a mutex closes it deterministically for every schedule - which is why "we never saw interleaving" is not an argument against the mutex.

The mutex also bought priority inheritance on the serial path: without it, a priority-1 holder could delay a priority-2 (or -3) producer by the full transmit time with nothing bounding the inversion.

7.4 Trade-offs

- Length-1 overwrite queues discard history.** Only the newest sample survives; a consumer that was busy misses the intermediate values. Accepted because both consumers are *level* consumers (render the current value, evaluate the current temperature), not event loggers.
- Bounded 50 ms display wait vs. power.** `DisplayTask` wakes 20 times a second even when nothing changed. Accepted: the wake costs one queue check, and the alternative (unbounded wait) would delay page turns by up to the next 2 s sample.
- Mutex around the whole line** blocks a caller for up to ~1.3 ms (15 bytes at 115200 baud). Accepted because correctness of the log is the verification instrument; a per-byte lock would not fit the HAL's API anyway.
- Polling input/motion** instead of interrupts: GPIO interrupts would cut latency, but edge decode, debounce and the Wokwi waveform all work at the poll rate, and polling keeps every state transition inside a task context where it can be reasoned about (and where section 19's blocking rule applies uniformly).
- Argued vs. measured utilisation.** `configGENERATE_RUN_TIME_STATS` is 0, so CPU claims rest on blocking budgets, not on a profiler. The fault experiments are the partial substitute: they show what happens when a budget is removed, even though they do not add up to a utilisation figure.

7.5 Alternative approaches rejected

Alternative	Why it was rejected
One shared sensor queue for both consumers	a queue hands each item to exactly one receiver, so <code>DisplayTask</code> and <code>AlarmTask</code> would race and each see half the stream; section 25's diagram is realised as one queue <i>per</i> consumer (<code>include/rtos_objects.h:35-49</code>)
Task notification for <code>InputTask</code> (the spec example's other option)	notifications carry no payload; "latest page wins" would need a shared variable plus a lock, which reintroduces exactly the unsynchronised shared state section 25 forbids
Binary semaphore instead of a mutex for the UART	no owner means no priority inheritance, i.e. the unbounded inversion class the mutex exists to prevent (<code>include/rtos_objects.h:166-175</code>)
A display lock instead of single-owner OLED	a lock around I2C1 would be a second mechanism protecting a resource that only one task ever touches; ownership gives the same guarantee with zero runtime cost (section 26)
<code>portMAX_DELAY</code> on <code>DisplayTask</code> 's receive	correct while samples were the only screen change; after milestone 8 a page turn must also wake it, so the wait had to become bounded (<code>src/display.cpp:54-66</code>)
Hand-rolled fixed-point formatting instead of <code>-wl, -u, _printf_float</code>	would spread rounding decisions through display and serial code; the float formatter costs flash but keeps <code>%f</code> in one honest place
3000 ms sensor period to fix DHT22 failures	measured to make the failure rate <i>worse</i> (section 7.2) - the rejected hypothesis that shaped the real fix
1 ms input poll	measured to stall the Wokwi emulator (< 1 s of output in a 45 s run); 2 ms is the fastest period that keeps the simulation real-time (<code>src/input.cpp:30-49</code>)
clang-tidy for static analysis	needs a compilation database for the STM32Cube framework that this project does not generate; its findings would describe the build setup, not the application (<code>docs/static-analysis.md</code> section 1)
Editing the vendor <code>port.c</code> in place	a framework package update would silently revert it; the patch is instead re-derived at build time and aborts if the vendor text moves (<code>scripts/freertos_build.py</code>)
Livelock fix by simply lowering <code>InputTask</code> 's priority	would hide the emulator defect rather than fix it, and would break the section 39 justification for priority 3 (edges cannot be replayed)

7.6 Limitations

Consistent with the README's *Limitations* section:

- **Simulation fidelity.** Everything runs in Wokwi: the DHT22, LDR, PIR and encoder are models with constant outputs unless a scenario changes them. No claim is made about real sensor noise, real buzzer acoustics, or timing that only the simulator tolerates (e.g. the 5 ms scheduler suspension during a DHT22 frame) - those must be re-measured on silicon.
- **Four of ten functional checks used synthetic stimulus** (FT-04, FT-05, FT-08, FT-10): the firmware path is proven, the physical element is not.
- **No PWM buzzer alarm** - the output is a DC level; no acoustic claim is made anywhere in this repository.
- **No encoder/PIR automation headless** - see above; manual follow-up in the Wokwi web UI is still recommended.
- **No real-time statistics** - utilisation is argued, not measured.
- **No hardware-in-the-loop run** - `pio run -t upload` exists but was never executed; the laboratory scopes the deliverable to simulation.
- **The Wokwi port patch is untested on real silicon** - architecturally neutral by construction, but not executed on a physical Blue Pill.
- **USART1 RX is configured but unused**; the verification input path is the simulator's scenario control, not a command console.

8. Conclusion

8.1 What was learned

1. **Priority is scheduling urgency, and it is falsifiable.** The section 39 argument (prompt response / tolerated latency / what breaks if wrong) was turned into measurements: removing one blocking line starves the system (5 lines instead of 216), and promoting one task costs the alarm 111 ms. Priorities stopped being numbers chosen by feel and became claims that an experiment can refute.
2. **The synchronisation primitive must match the data's shape.** Samples are levels, so length-1 overwrite queues; state is a level, so event bits with non-destructive reads; the UART is a resource, so a mutex with priority inheritance. Choosing by habit (one queue for everything, or a semaphore because "it also locks") would have produced real failure modes - split consumer streams, unbounded inversion.
3. **Shared-resource bugs can be silent.** The `HAL_BUSY` defect dropped whole lines with no error, no garbling and no assertion; only counting message shapes against a reference exposed it. Verification has to compare against an expected *distribution*, not just check

that something was printed.

4. **Hypotheses need measured signs.** The 3000 ms DHT22 experiment made things worse and thereby ruled out the most obvious explanation; the 1 ms input poll stalled the emulator and ruled out the naive arithmetic answer. Both rejections are documented because they are the reason the final values (10 ms start pulse, 2 ms poll) are defensible rather than lucky.
5. **Emulators are part of the target.** Two instruction-level quirks (PendSV serviced at the ICSR store, `cpsie` decoded inverted) had to be worked around without making the firmware un-portable; the discipline that made this survivable was documenting the patch where it is applied and making the build fail if the vendor text changes.
6. **Structure is what makes verification cheap.** Pure decision functions at the centre meant 29 unit tests ran on the desktop in under 4 s; one module per subsystem meant a static-analysis finding pointed at exactly one file; one serial choke point meant the mutex was added once.

8.2 What should be improved

Short, and consistent with the README's *Limitations / Future Improvements*:

- Enable runtime CPU statistics (`configGENERATE_RUN_TIME_STATS`) to replace the argued-but-unmeasured utilisation claims.
- Add a watchdog task asserting that each task reported in within its budget, turning the blocking-budget table into an executable assertion.
- Run hardware-in-the-loop on a physical Blue Pill - which would also test the Wokwi port patch on real silicon and re-measure the DHT22 timing margins.
- Replace the LDR relative-percentage curve with calibrated lux and store the calibration; add a command protocol on the existing USART1 RX for thresholds and page selection.
- Drive the buzzer with PWM so the alarm is a tone rather than a level, if the assignment's acoustic intent is ever in scope.
- Repeat FT-04/FT-05/FT-08/FT-10 by hand in the Wokwi web UI (and on hardware) to close the synthetic-stimulus gap.

Appendix A. Requirements Traceability Matrix (section 60)

Placement choice: the matrix lives in this appendix rather than inside section 5, so that section 5 documents *method and results* while the matrix stays a flat table that can be checked row by row against the specification. Every verification entry names evidence that exists in this repository; no row is left unresolved. Where coverage is partial, the row says so.

A.1 Functional requirements (section 4)

Requirement	Implementation	Verification
FR-01 temperature measurement	<code>src/sensors.cpp</code> - <code>SensorTask</code> (DHT22 frame, 2000 ms <code>vTaskDelayUntil</code>)	FT-01 (<code>docs/functional-verification.md</code> section 2, run A lines 5/27/48/71); 30 <code>Temperature</code> : lines per 60 s reference run
FR-02 humidity measurement	<code>src/sensors.cpp</code> - <code>SensorTask</code> (same frame, same publish)	FT-02 (run A lines 6/28/72); 30 <code>Humidity</code> : lines per 60 s reference run
FR-03 ambient-light measurement	<code>src/sensors.cpp</code> - <code>SensorTask</code> (ADC1_IN0, <code>ldr_to_light_percent()</code>)	FT-03 (run A lines 7/29/73); 30 <code>Light_level</code> : lines per 60 s reference run
FR-04 motion detection	<code>src/motion.cpp</code> - <code>MotionTask</code> (PB8, 10 ms poll, sets <code>EVENT_MOTION</code>)	FT-08 and FT-10 (VCD <code>PIR_STIM</code> pulses 1.200 s / 6.000 s); <code>Motion</code> : <code>detected</code> line; synthetic PIR edge disclosed in <code>docs/functional-verification.md</code> section 4
FR-05 OLED display, one measurement at a time	<code>src/display.cpp</code> - <code>DisplayTask</code> , sole owner of I2C1, four pages via <code>render_page()</code>	FT-01-FT-03 (serial values plus OLED screenshots <code>31.00 C ALARM</code> , <code>45.00 %</code> , <code>38 %</code>); <code>Page</code> : line once per run
FR-06 encoder navigation	<code>src/input.cpp</code> - <code>InputTask</code> ; <code>nextDisplayMode()</code> / <code>previousDisplayMode()</code> in <code>include/input.h</code>	<code>pio test/test_navigation</code> x11 (29/29 total); FT-04 and FT-05 (8 detents -> 8 page changes incl. wrap, VCD <code>ENC_CLK</code> / <code>ENC_DT</code>); synthetic detents disclosed
FR-07 temperature alarm	<code>src/alarm.cpp</code> - <code>AlarmTask</code> + pure <code>evaluateTemperature()</code> (18.0/30.0 inclusive), buzzer PB0	<code>pio test/test_alarm</code> x10; FT-06 and FT-07 (<code>Alarm</code> : <code>HIGH_TEMPERATURE</code> / <code>NORMAL</code> , VCD rise 6.252 s / fall 12.252 s, <code>expect-pin</code> assertions passed)
FR-08 activity state ACTIVE/INACTIVE	<code>src/system_state.cpp</code> - <code>StateTask</code> ; <code>evaluateSystemState()</code> in <code>include/system_state.h</code>	<code>pio test/test_state</code> x8; <code>State</code> : <code>ACTIVE</code> printed once at boot in every run
FR-09 automatic inactivity (15 s)	<code>src/system_state.cpp</code> - <code>StateTask</code> , <code>SYSTEM_INACTIVE_TIMEOUT_MS</code> = 15000, timeout branch of <code>xEventGroupWaitBits</code>	FT-09 (<code>State</code> : <code>ACTIVE</code> line 3 -> <code>State</code> : <code>INACTIVE</code> line 61 in run A; line 78 in run B); 1 <code>INACTIVE</code> per 60 s reference run
FR-10 automatic reactivation	<code>src/motion.cpp</code> - <code>MotionTask</code> sets <code>EVENT_MOTION</code> ; <code>src/system_state.cpp</code> consumes it with clear-on-exit	FT-10 (<code>Motion</code> : <code>detected</code> line 93 immediately followed by <code>State</code> : <code>ACTIVE</code> line 94, then the sensor burst resumes); synthetic PIR edge disclosed

A.2 Required FreeRTOS concepts (section 9)

Requirement	Implementation	Verification
Multiple tasks	8 tasks created in <code>src/main.cpp:216-223</code>	60 s run: all eight make progress (Task A/B 60/60, sensors 30/30/30, Page: / Alarm: / State: 1/1/1+1); the converse is measured by <code>docs/fault-experiments.md</code> section 49 (5 lines when one task stops blocking)
Explicit task priorities	<code>src/main.cpp:104-215</code> (section 38/39 table) and the <code>xTaskCreate</code> calls; README <i>Task Design</i>	section 59 table and per-row justification in this report; <code>docs/fault-experiments.md</code> section 50 measures the effect of changing one
Blocking delays	<code>vTaskDelay()</code> in <code>TaskA/TaskB</code> (1000 ms), <code>InputTask</code> (2 ms), <code>MotionTask</code> (10 ms)	section 49 fault experiment: deleting <code>InputTask</code> 's delay -> 5 lines vs 216, Task A/B 0/0
<code>vTaskDelayUntil()</code> for at least one periodic task	<code>src/sensors.cpp:448</code> - <code>SensorTask</code> , 2000 ms absolute period	30 samples per 60 s in every reference run (<code>docs/functional-verification.md</code> section 5); source compiles under <code>pio run</code>
At least one queue	<code>displayQueue</code> , <code>alarmQueue</code> , <code>modeQueue</code> created in <code>src/rtos_objects.cpp</code> (length 1, <code>xQueueOverwrite</code>)	FT-01-FT-05 each crosses at least one queue end-to-end; <code>docs/fault-experiments.md</code> section 50 shows the producer is never blocked by the consumer (line counts unchanged while the display is promoted)
At least one mutex	<code>serialMutex = xSemaphoreCreateMutex()</code> in <code>src/rtos_objects.cpp:79</code> , taken in <code>serial_write()</code>	<code>docs/fault-experiments.md</code> section 51: 215 vs 216 lines with Page: Temperature lost, twice; the before/after record on <code>include/rtos_objects.h:148-164</code> (209 of 216 lines without it)
At least one event group or task notification	<code>systemEvents</code> with <code>EVENT_ACTIVE / EVENT_MOTION / EVENT_ALARM</code> (<code>include/rtos_objects.h:82-84</code>)	FT-08 (encoder gated on <code>EVENT_ACTIVE</code> still works), FT-09/FT-10 (the <code>State:</code> transitions), <code>docs/functional-verification.md</code> sections 3.4 and 2
A state machine	<code>ACTIVE/INACTIVE</code> in <code>src/system_state.cpp</code> ; pure <code>evaluateSystemState()</code> in <code>include/system_state.h</code>	<code>pio test/test_state x8</code> ; FT-09 and FT-10
Inter-task communication	3 queues + 3 event bits (README <i>Inter-Task Communication</i> ; Figure 3)	FT-01-FT-10: every row's path runs sensor/input -> queue or bit -> consumer
Shared-resource protection	<code>serialMutex</code> for USART1 (section 36); single-owner OLED for I2C1 (section 26)	fault experiment 3 (mutex removed -> line lost); OLED ownership holds trivially - <code>DisplayTask</code> is the only writer of I2C1 (grep-able: no other module includes the display driver's internals)

A.3 Submission checklist rows not covered above (section 65)

Category	Requirement	Implementation	Verification
Build	<code>pio run</code> succeeds	<code>platformio.ini</code> + <code>scripts/freertos_build.py</code>	<code>pio run</code> SUCCESS, Flash 48.2 % (31588/65536), RAM 55.4 % (11348/20480) - unchanged across milestones
Simulation	DHT22, LDR, PIR, encoder, OLED, buzzer; Wokwi operational	<code>diagram.json</code> , <code>wokwi.toml</code>	<code>wokwi-cli lint</code> clean; 60 s headless run 216 lines (this milestone); runs A/B/A2/A3/D1/D2 in <code>docs/functional-verification.md</code>
Testing	at least 13 meaningful unit tests; <code>pio test</code> succeeds	<code>test/test_alarm</code> , <code>test/test_navigation</code> , <code>test/test_state</code>	<code>pio test</code> -> 29/29 (10 + 11 + 8)
Testing	functional verification table completed	scenario runs documented outside the repo; <code>diagram.json</code> untouched	<code>docs/functional-verification.md</code> section 2 (FT-01...FT-10, all PASS with observed values)
Testing	fault experiments completed	one-line changes in <code>src/input.cpp</code> , <code>src/main.cpp</code> , <code>src/rtos_objects.cpp</code> , each reverted	<code>docs/fault-experiments.md</code> sections 1-4 (5 lines / +111 ms / 215 lines, restoration proven)
Quality	<code>pio check</code> completed; findings analysed	flags pinned in <code>platformio.ini [env]</code>	<code>pio check</code> 0 high / 0 medium / 35 low; <code>docs/static-analysis.md</code> triages all 38 findings row by row
GitHub	meaningful history, professional README, architecture diagrams, no build artifacts	30 commits M0-M16 with milestone messages; README.md; docs/images/ Figures 1-5; .pio/ ignored	<code>git log --oneline</code> (no bulk commit); README section list per section 55; <code>git status --short</code> shows only intended files
Report	laboratory report, FreeRTOS task table, traceability matrix, test + static-analysis discussion, limitations	<code>docs/laboratory-report.pdf</code> from <code>docs/laboratory-report.md</code> (this document)	section 3 (section 59 table), Appendix A (section 60 matrix), sections 5-7, section 7.6
Portfolio	Hackster.io article, GitHub link and attribution	not yet implemented - scheduled as milestone 17 (sections 61-63); nothing in this repository claims it exists	none; the README makes no publication claim

Source of this report: `docs/Laboratory-report.md`. To regenerate the PDF: render the Markdown to HTML with the Python `markdown` package (extensions `tables`, `fenced_code`), then print it with headless Chrome, e.g. `chrome.exe --headless --print-to-pdf=docs/Laboratory-report.pdf report.html`. Neither step touches `platformio.ini` or the firmware build.